

Practical 2:

Linear and Binary Search: Iterative and Recursive Approaches

Practical Aim

To implement linear and binary search using both iterative and recursive approaches.

Problem 1

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found. Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

Code:

```
import java.util.Scanner;

public class VehicleSearch {

    static int recursiveSearch(int[] arr, int x, int index) {
        if (index == arr.length) {
            return -1;
        }

        if (arr[index] == x) {
            return index + 1;
        }

        return recursiveSearch(arr, x, index + 1);
    }
}
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the number of vehicles parked: ");
    int n = sc.nextInt();

    int[] arr = new int[n];

    System.out.print("Enter the Vehicle Plate Numbers: ");
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }

    System.out.print("Enter the Plate Number you want to search: ");
    int x = sc.nextInt();

    int iterativePosition = -1;

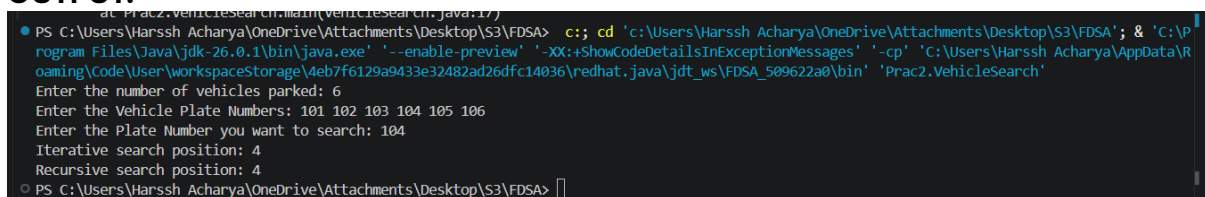
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            iterativePosition = i + 1;
            break;
        }
    }

    int recursivePosition = recursiveSearch(arr, x, 0);

    System.out.println("Iterative search position: " + iterativePosition);
    System.out.println("Recursive search position: " + recursivePosition);
}
}

```

OUTPUT:



```

PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA> cd 'c:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\4eb7f6129a9433e32482ad26dfc14036\redhat.java\jdt_ws\FDSA_509622a0\bin' 'Prac2.VehicleSearch'
Enter the number of vehicles parked: 6
Enter the Vehicle Plate Numbers: 101 102 103 104 105 106
Enter the Plate Number you want to search: 104
Iterative search position: 4
Recursive search position: 4
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA>

```

Conclusion:

I learned how to search for a vehicle plate number using both iterative and recursive methods. Both methods helped me understand searching in arrays.

Problem 2

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code. Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

CODE:

```
package Prac2;

import java.util.Scanner;

public class LinearSearchint {

    static int recursiveSearch(int arr[], int x, int index) {

        if (index == arr.length)
            return -1;

        if (arr[index] == x)
            return index + 1;

        return recursiveSearch(arr, x, index + 1);
    }

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of books: ");
        int n = sc.nextInt();

        int arr[] = new int[n];

        System.out.print("Enter sorted book codes: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter book code to search: ");
        int x = sc.nextInt();

        int loopPosition = -1;
```

```
for (int i = 0; i < arr.length; i++) {  
    if (arr[i] == x) {  
        loopPosition = i + 1;  
        break;  
    }  
}  
  
int recursivePosition = recursiveSearch(arr, x, 0);  
  
System.out.println("Loop method position: " + loopPosition);  
System.out.println("Recursive method position: " + recursivePosition);  
  
sc.close();  
}  
}
```

OUTPUT:

```
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\4eb7f6129a9433e32482ad26dfc14036\redhat.java\jdt_ws\FDSA_509622a0\bin' 'Prac2.LinearSearchint'  
Enter number of books: 5  
Enter sorted book codes: 4 5 6 7 8  
Enter book code to search: 6  
Loop method position: 3  
Recursive method position: 3  
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA> █
```

Conclusion:

I learned how to search an element in an array using both loop and recursive methods. This helped me understand the difference between the two searching methods.

Github Link: <https://github.com/haarshaacharya/FDSA/tree/main/Prac2>